



RELICHECK · ENGINEERING REFERENCE

MM Studio Bible

The single reference for ReliCheck Mixed-Methods Studio

v2 wizard build · closed-beta launch · 2026-05-20

The single reference for ReliCheck Mixed-Methods Studio. Last updated 2026-05-20 at the close of the v2 wizard + closed-beta launch build.

This document is for whoever picks up MM Studio next: another developer, a designer, a future-Don, or Claude in a fresh session with no context. Read top to bottom once. Come back to specific sections when you need them.



Table of contents

- 1 What MM Studio is
- 2 Who it is for
- 3 The product surface
- 4 The v2 user flow
- 5 The 20-step internal pipeline
- 6 The data model
- 7 The API surface
- 8 The front-end (app-2026.html)
- 9 The admin surface
- 10 The closed-beta cohort plumbing
- 11 Operational conventions and "rules"
- 12 Known limits and edge cases
- 13 What is shipped vs. preview
- 14 Path forward after closed beta

1. What MM Studio is

Mixed-Methods Studio is a feature inside ReliCheck App 2.0 that takes survey data combining closed-ended items (Likert scales, ratings, demographics) with open-ended comments and walks the researcher through coding, categorizing, integrating, and reporting on it. The work that traditionally lives across NVivo, SPSS, and Word collapses into one flow inside one product.

The user brings a dataset. The studio writes themes from the open-ended responses, codes every row with a theme and a sentiment, builds derived variables alongside the closed-ended items, drafts integration paragraphs that bring quant and qual together, runs a methodological audit, and exports a draft report in DOCX or PDF.

The studio recommends. The user accepts or adjusts. Themes can be renamed, merged, recoded. Suggested analyses can be ignored. Narrative drafts can be rewritten. The author keeps editorial control at every step. This matters for citation and for trust.

MM Studio is currently in **closed beta** for the ReliCheck closed-beta cohort (dissertation-level researchers and mixed-methods faculty). The endpoint behind it is reachable today by anyone with the `?draft=1` URL flag, but the navigation entry is hidden for users without that flag.

2. Who it is for

The target audience for the closed beta is:

- Dissertation-level researchers actively coding open-ended responses for a study
- Mixed-methods faculty who teach and practice the methodology
- Program evaluation researchers who need a defensible integrated findings section

The target audience for general availability (post-beta) is broader: doctoral students, education and HR practitioners, accreditation reviewers, policy researchers. The studio's design pedagogy was tuned to dissertation-level methods knowledge: the user knows what a Likert scale is, what "compare groups" means, and what an integrated findings section looks like. The studio does not teach these. It assumes them.

The studio's pedagogy is **data structure first, intent second, analysis third**. The wizard reflects this order:

- 1 Title the study (frame it as a study, not a spreadsheet)
- 2 Bring the data in and confirm what the studio read
- 3 Then frame what you want to learn (data kind + intent + chosen design)
- 4 Then do the analysis

This is the right order for non-stats audiences. Asking "what are you trying to understand?" before knowing what data is in hand leads users to ask questions their data cannot answer. Starting with data structure forces an honest reckoning with what is actually available.



3. The product surface

MM Studio lives at `app-2026.html?draft=1#/mm`. The sidebar entry says "MM Studio (DRAFT)" and is hidden unless `?draft=1` is on the URL.

The four screens a user sees:

Studio dashboard. Purple hero with the welcome banner. Four Quick Action cards (Start a new project, Continue last project, Load a sample project, Browse all projects). Recent projects list. Recent activity feed. Beta guide link.

Project wizard. Three outer steps: Title your study, Your data (with two sub-steps: intake then confirm-mapping), Frame your study (with three sub-steps: data kind, intent, design). Header bar holds the Back, stepper, and Next buttons; no bottom button bar.

Project shell. After the wizard completes (or for a legacy project from before the v2 migration), the user lands here. A "What happens here" walkthrough card sits at the top, then a row of stat cards (Responses / Categories / Coded / Status), a three-stage strip (Confirm your data, Run the Builder, Analyze and report), and the active stage's content below. The eight Analyze tabs live inside the third stage.

Analyze tabs. Categories, Dataset, Analysis, Joint Display, Integration, Strength Check, Report. On the scores+comments pathway, two extra tabs appear: Score-to-Theme and Alignment. A "Recommended Order" strip above the tab row guides first-time users in pedagogical order; each tab is independently clickable.

A floating **Feedback** button sits in the bottom-right of every MM Studio screen so beta users can send notes in context.



4. The v2 user flow

This is the flow as of the closed-beta launch. The legacy "20-step" implementation lives in the data model and APIs (the underlying pipeline never went away). The wizard re-frames intake and framing in three outer steps; the post-wizard work uses the existing three-stage project shell.

Wizard Step 1: Title your study

- Required: study title (max 200 chars)
- Optional: description (max 2000 chars)
- On Next: a `mm_projects` row is created with `wizard_step = 1` (after the advance, 2). Subsequent steps update the existing row instead of creating new ones.

Validation: empty title fails inline and toast. By design, users must name the study before moving on. This is the pedagogy ritual — start with the study, not the data.

Wizard Step 2: Your data

Two sub-steps shown one at a time with a chip strip across the top.

Sub 2.1 — Bring your data in. Four routes:

- **Upload a file** with drag-and-drop or file picker. Accepts .csv, .tsv, .xlsx, .xls. Parsed entirely client-side via `mmParseFile` to get headers + rows before anything hits the server.
- **Paste responses** for quick tests.
- **Link an existing ReliCheck survey** — uses the user's existing surveys list.
- **I do not have data yet** — opens a modal with two choices: open the ReliCheck survey builder (the project is already saved as a draft and waits) or save the project and return to the Studio dashboard.

Sub 2.2 — Confirm what we found. The column mapper. Multi-select on each role:

- Response ID column (single)
- Question / item identifier (single, optional)
- Open-ended question columns (multi)
- Closed-ended survey variables (multi)
- Demographic / grouping variables (multi)
- Outcome variables (multi)
- Time / wave variables (multi)

On Next: the studio runs the real ingest (`/api/mm/ingest.php` or `/api/mm/link-survey.php` depending on intake type), then `/api/mm/wizard.php` saves the field map, then `/api/mm/framing.php` advances `wizard_step` to 4 server-side.

Wizard Step 3: Frame your study

Three sub-steps shown one at a time.

Sub 3.1 — What kind of data do you have? Five multi-select options:

- Open-ended responses only
- Survey data with open-ended responses
- Survey data and separate interview / focus group data
- Quantitative data only, but I want to add qualitative interpretation
- I want to build a mixed-methods study from scratch

On Next: PATCH `framing.data_kinds`, sets `framing_status = in_progress`.

Sub 3.2 — What are you trying to understand? Eight multi-select options:

- Explain survey results
- Find themes in open-ended responses
- Compare groups
- Build variables from text
- Strengthen a report with qualitative evidence
- Create a mixed-methods findings section
- Prepare evaluation or accreditation evidence
- Explore patterns before building a survey

On Next: PATCH `framing.intent_purposes`, then immediately GET `/api/mm/design-recommendations.php` to populate the badges for sub-3.3.

Sub 3.3 — Choose your mixed-methods design. Five single-pick options:

- A. Explain the numbers with comments
- B. Turn comments into measurable support
- C. Compare themes across groups
- D. Build variables from open-ended data
- E. Create a full integrated mixed-methods report

Server-driven "Recommended" badges based on Steps 2 mapping + 3a + 3b. Multiple badges are possible.

On "Start project" (final button): PATCH `framing.chosen_design`, sets `framing_status = complete`, advances `wizard_step` to 99. Then navigates the user to the project shell.

Post-wizard: project shell

The walkthrough card explains three stages:

- 1 **Confirm your data** — quality brief, data integrity preview
- 2 **Run the Builder** — review and clean, two-pass theme discovery, batched coding
- 3 **Analyze and report** — the eight tabs

The user picks a stage button to jump there. Stage 3 hosts the tab strip with the "Recommended Order" guidance.

Resume behavior

If a user closes the browser mid-wizard and comes back, the dashboard's project list shows the in-progress project. Clicking it calls `mmRenderProject`, which fetches `/api/mm/framing.php` and uses the saved `wizard_step` + `framing` fields to drop the user back at the right sub-screen. Legacy projects (`framing_status = skipped_legacy`, set by the Phase 170 migration) bypass the wizard entirely and go straight to the project shell.



5. The 20-step internal pipeline

The wizard re-frames the user experience; the underlying pipeline still has 20 phases. They survive because the analysis tabs were built against them. Knowing the phase numbers helps when reading old schema files and endpoints.

- 1 **Phase 1-5** (Checkpoint 1): project model, intake routes, file parsing, dataset preview, simple-step flow.
- 2 **Phase 6**: Open-Ended Response Quality Brief — flags blank, short, duplicate, low-effort responses.
- 3 **Phase 7**: Per-question themes — uses the `question_id` column to group theme discovery per question.
- 4 **Phase 8-9**: Response-level coding and recoding.
- 5 **Phase 10**: Variable Builder — derived variables (Theme presence, Theme intensity, Sentiment label, Sentiment numeric, Response length chars, Response length words).
- 6 **Phase 11**: IV/DV builder — assign predictor/outcome roles to variables for analysis recommendations.
- 7 **Phase 12**: Clustering — group correlated themes into higher-order categories.
- 8 **Phase 13**: Variable rename + notes + inline cell edit + Excel + SPSS export.

- 9 **Phase 14:** Analysis suggest + run — recommends correlations, t-tests, chi-squares, ANOVAs, regressions based on variable types. Math runs through the stats helper library.
- 10 **Phase 15:** Joint Display — the canonical mixed-methods table reviewers expect.
- 11 **Phase 16:** Integration paragraphs — drafted prose per theme that brings quant and qual together.
- 12 **Phase 17:** Strength Check — methodological audit (sample sizes, theme coverage, alignment depth).
- 13 **Phase 18:** Report — sectioned editable document with template, sections, body text/html, version tracking.
- 14 **Phase 19:** Rich-text section editor + version history + section notes + AI rewrite bubble.
- 15 **Phase 20:** Report export — DOCX (editable) and PDF (read-only). Templates (Phase 20 Phase E) let the user save a finished project's themes + variable roles as a template they can apply to new projects.

Phase 155 introduced the dedicated `mm_*` tables (versus shoehorning into existing tables). **Phase 156-167** are scattered iterations. **Phase 170** is the v2 wizard restructure (framing table + wizard_step). **Phase 171** is the closed-beta cohort flag on `promo_codes`.



6. The data model

All MM Studio tables live in the canonical ReliCheck database `dbs15641829` and are prefixed `mm_`. Created in `db/schema_phase155.sql` (the master MM schema), then iteratively extended.

Core tables

`mm_projects` (Phase 155, extended Phase 170)

The root object. One row per project.

```

id BIGINT UNSIGNED PK
user_id BIGINT UNSIGNED -- owner. Note: NOT owner_id like surveys.
title VARCHAR(200)
description TEXT NULL -- Phase 170, optional study description
pathway ENUM('scores_plus_comments', 'comments_only')
survey_id VARCHAR(64) NULL
dataset_id BIGINT UNSIGNED NULL
status ENUM('draft', 'active', 'archived')
wizard_step TINYINT UNSIGNED -- Phase 170, 1..99
data_kinds TEXT NULL -- JSON array, legacy field
notes MEDIUMTEXT NULL
created_at DATETIME
updated_at DATETIME

```

mm_project_framing (Phase 170)

One row per project, keyed on `project_id`. Holds the v2 wizard's framing fields.

```

project_id BIGINT UNSIGNED PK / FK -> mm_projects(id) ON DELETE CASCADE
data_kinds TEXT NULL -- JSON array of canonical slugs
intent_purposes TEXT NULL -- JSON array of canonical slugs
chosen_design VARCHAR(40) NULL -- design_a..design_e
framing_status ENUM('pending', 'in_progress', 'complete', 'skipped_legacy')
created_at DATETIME
updated_at DATETIME

```

The canonical slugs are documented in `api/mm/framing.php`:

- `data_kinds`: `open_only`, `survey_plus_open`, `survey_plus_interviews`, `quant_plus_interpretation`, `build_from_scratch`
- `intent_purposes`: `explain_survey`, `find_themes`, `compare_groups`, `build_variables`, `strengthen_report`, `findings_section`, `eval_evidence`, `explore_patterns`
- `chosen_design`: `design_a`, `design_b`, `design_c`, `design_d`, `design_e`

The wizard UI uses legacy slugs (e.g. `open_ended_only`) and the framing endpoint maps both ways. New code should use the canonical short slugs.

Data tables

mm_data_sources — one row per intake (upload, paste, linked survey). A project can have multiple.

mm_text_responses — one row per open-ended response. The unit of analysis throughout the studio.

```
id BIGINT UNSIGNED PK
project_id BIGINT UNSIGNED FK
source_id BIGINT UNSIGNED FK
respondent_ref VARCHAR(120) -- the user's response ID column
question_id_raw VARCHAR(64) NULL -- Phase 157, per-question grouping
question_text_raw VARCHAR(500) NULL -- Phase 157, question prompt text
group_value VARCHAR(200) NULL -- demographic / grouping value
numeric_value DECIMAL(10,4) NULL -- the associated score
text MEDIUMTEXT
created_at DATETIME
```

Analytical tables

mm_extracted_concepts — short key terms/phrases pulled from texts.

mm_theme_categories — themes the Builder discovered. Editable by the user.

```
id BIGINT UNSIGNED PK
project_id BIGINT UNSIGNED FK
name VARCHAR(200)
description VARCHAR(600) NULL
source_mode ENUM('auto','guided','hybrid','user')
confidence ENUM('high','moderate','low')
position INT UNSIGNED -- display order
```

mm_coding_rules — keyword / phrase / regex rules that map text → category. Optional.

mm_coded_responses — response-to-category assignments. One row per (response_id, category_id) pair. Unique key on the pair.

mm_sentiment_scores — per-response sentiment (positive/neutral/negative/mixed) plus optional indicators JSON (frustration, trust, urgency, etc.).

mm_generated_variables — derived variables (theme presence, intensity, sentiment, response length).

mm_structured_datasets + **mm_dataset_cells** — the exportable dataset table. Schema in JSON, cells in narrow rows so the table stays wide enough for any project.

mm_evidence_alignment_results — Evidence Alignment Check rows.

mm_group_voice_results — theme distribution per group value.

mm_evidence_matrices — Joint Display rows.

mm_follow_up_questions — questions the Follow-Up Builder generates.

Report tables

mm_report_sections (Phase 162) — one row per (project_id, section_key) with body_text + body_html.

mm_report_section_versions (Phase 19 Phase B) — version history per section, append-only.

mm_report_section_notes (Phase 19 Phase C) — todo-style notes pinned to sections.

Templates

mm_project_templates (Phase 20 Phase E1) — themes + variable roles saved from a finished project for reuse on a new project.

Beta

mm_feedback (Phase 169) — in-app feedback submissions from the floating Feedback widget. Fields: user_id, project_id, rating (1-5), comment, page_kind, user_agent, viewport, status, admin_note, timestamps.



7. The API surface

All MM Studio endpoints live under `/api/mm/` except where noted. All require an authenticated session (`require_auth()`). All accept JSON bodies and return JSON. All check origin via `check_origin()` on writes.

Project lifecycle

- `GET /api/mm/projects.php` — list current user's projects
- `POST /api/mm/projects.php` — create a project (title required, description optional, pathway, data_kinds, survey_id, dataset_id, notes)
- `GET /api/mm/project.php?id=N` — fetch one project + counts
- `PATCH /api/mm/project.php` — update title / notes / status
- `DELETE /api/mm/project.php` — delete (cascades to all `mm_*` child tables)

v2 wizard

- `GET /api/mm/framing.php?project_id=N` — fetch framing row + project's wizard_step + canonical slug lists
- `PATCH /api/mm/framing.php` — upsert framing fields, optionally advance wizard_step
- `GET /api/mm/design-recommendations.php?project_id=N` — pure scoring logic returning A-E with recommended boolean per design

Intake

- `POST /api/mm/ingest.php` — bulk insert text responses with respondent_ref, group_value, numeric_value, question_id_raw, question_text_raw

- `POST /api/mm/link-survey.php` — bind a ReliCheck survey to the project, ingest the responses
- `POST /api/mm/wizard.php` — save legacy wizard step state (kept for compatibility)

Quality + build

- `POST /api/mm/quality-brief.php` — run the quality scan
- `POST /api/mm/seed.php` — submit category seeds for Hybrid/Guided builder modes
- `POST /api/mm/build.php` — the two-pass theme builder
- `GET /api/mm/responses.php?project_id=N` — fetch responses for review
- `PATCH /api/mm/responses.php` — edit a response
- `DELETE /api/mm/responses.php` — delete a response
- `POST /api/mm/themes-per-question.php` — per-question theme generation
- `POST /api/mm/code-existing.php` — apply themes to responses

Categories and clustering

- `GET /api/mm/categories.php` — list categories
- `POST /api/mm/recode-category.php` — recode a single category
- `POST /api/mm/clusters.php` — generate higher-order clusters

Coded responses

- `GET /api/mm/coded-responses.php?project_id=N` — paged list of coded responses for review

Dataset / variables

- `POST /api/mm/dataset.php` — build the structured dataset
- `PATCH /api/mm/dataset-cell.php` — inline cell edit
- `POST /api/mm/variable-roles.php` — assign predictor/outcome/either roles

Analysis + integration

- `POST /api/mm/analysis-suggest.php` — recommend IV/DV pairs and tests
- `POST /api/mm/analysis-run.php` — run a specific test
- `POST /api/mm/joint-display.php` — build the joint display
- `POST /api/mm/integration.php` — generate / save / clear integration paragraphs
- `POST /api/mm/score-to-theme.php` — scores + comments pathway only
- `POST /api/mm/alignment.php` — scores + comments pathway only
- `POST /api/mm/matrix.php` — evidence matrix rows

Strength + report

- `POST /api/mm/strength-check.php` — methodological audit
- `GET /api/mm/report.php?project_id=N` — load report sections
- `POST /api/mm/report.php` — generate / save / reset / version / note actions
- `POST /api/mm/report-rewrite.php` — AI rewrite endpoint for selected text
- `GET /api/mm/report-export.php?format=json&project_id=N` — JSON export
- `POST /api/mm/report-docx.php` — server-side DOCX export
- `POST /api/mm/save-to-datasets.php` — push the coded dataset into the main Datasets list

Templates

- `GET /api/mm/templates.php` — list templates
- `POST /api/mm/templates.php` — save / delete / apply actions

Feedback (closed beta)

- `POST /api/mm/feedback.php` — write a feedback row from the floating widget

Admin

- `GET /api/admin/mm/activity.php` — aggregate counters + per-user roll-up of MM Studio activity
- `GET /api/admin/beta/cohort.php` — list beta cohort promo codes + roster
- `POST /api/admin/beta/cohort.php` — create / deactivate cohort codes
- `GET /api/admin/feedback/list.php` — beta feedback inbox

All admin endpoints require `require_admin()` and return a soft warning payload (not an HTML 500) when MM tables are missing.



8. The front-end (app-2026.html)

MM Studio's front-end is a single-page section of `app-2026.html`. The relevant pieces:

Route entry

Route name: `mm`. Handler: `renderMixedMethodsStudio(container, param)` in `app-2026.html`. The route is gated by `MM.draftFlag()` (the `?draft=1` URL flag). Without the flag, users see a "Mixed-Methods Studio is in DRAFT" splash with an "Open draft preview" button.

`param` decoding:

- No param → render the Studio dashboard (`mmRenderLanding`)
- `'new'` → render the wizard (`mmRenderWizard`)
- Numeric → load the project; if mid-wizard, resume the wizard; otherwise render the project shell (`mmRenderProject` → `mmRenderProjectShell`)

State

`MM.state.current` — the loaded project row `MM.state.counts` — responses / categories / coded
`MM.wizard` — wizard-in-progress state object: `step`, `projectId`, `projectTitle`, `projectDescription`, `dataKinds`, `purposes`, `designChoice`, `dataSubStep`, `frameSubStep`, `fileParsed`, `fieldMap`, `intake`, `designRecsCache`

Wizard structure

`mmWizardDraw(container)` — renders the wizard chrome (header with Back / stepper / Next, title strip below, body).

Step dispatchers:

- Step 1: `mmWizardStepTitle`
- Step 2: `mmWizardStepData` (renders the sub-stepper chip strip, dispatches to `mmWizardStep3` for intake sub-step or `mmWizardStep4` for confirm sub-step)
- Step 3: `mmWizardStepFrame` (renders the sub-stepper chip strip, dispatches to `mmWizardStep1` for data kind, `mmWizardStep2` for intent, `mmWizardStep5` for design)

The legacy `mmWizardStep1..Step5` names map to single-purpose renderers (data kind picker, intent picker, intake routes, confirm-mapping, design picker) that the new dispatchers reuse rather than rebuild.

`mmWizardAdvance(container, step)` — validates and POSTs to the server for each step. Branches on `dataSubStep` for Step 2 and `frameSubStep` for Step 3.

Project shell

`mmRenderProject(container, id)` — loads the project, decides between wizard resume or project shell render based on `framing_status` + `wizard_step`.

`mmRenderProjectShell(container)` — renders the "What happens here" walkthrough card, stat cards, three-stage strip, and the active stage's content. The walkthrough card is dismissible per project (`localStorage mm_walkthrough_dismissed_<id>`).

Eight-tab analyze view

`mmStep3Analyze(body)` — renders the Recommended Order chip strip + the eight tabs. Tab visits are tracked per project in `localStorage (mm_tabs_visited_<id>)` so the strip advances as the user works. Feedback context updates when each tab is clicked so beta notes include the page kind.

Feedback widget

Defined as the `MMFeedback` IIFE near the bottom of the `app-2026.html` script section. The floating button (`#mmFeedbackBtn`) is rendered into the body's HTML. `MMFeedback.show() / .hide()` toggle visibility. `MMFeedback.setContext(projectId, pageKind)` updates the auto-attached context. The main `render()` function calls `show/hide` based on whether the current route is `mm`.



9. The admin surface

Three admin views were added for the beta launch. All live in `admin.html` under the System nav.

Cron Health — `data-view="cron"`. Reads `/api/admin/cron/heartbeat.php`. Shows every registered cron job with last seen, last status, run/error counts, and a health badge (ok / warn / down / no runs yet).

MM Studio activity — `data-view="mm-activity"`. Reads `/api/admin/mm/activity.php`. Shows six counter tiles (Users, Projects, Active 7d, Stuck, Themes built, Reports started) and a per-user table (name + email, project counts, themes built, reports started, last activity, colored status badge).

Beta cohort — `data-view="beta-cohort"`. Reads `/api/admin/beta/cohort.php`. Top: create-a-code form with defaults (BETA-MMSTUDIO-2026A, Researcher tier, 180 days, 25 uses). Below: one card per cohort code with the code, signup URL, copy button, deactivate button, and roster table.

Each view has a Reload button in the header. None auto-poll.



10. The closed-beta cohort plumbing

The cohort signup-to-landing chain has four parts working in concert.

1. Promo code creation (admin → Beta cohort): admin generates a promo code with `is_beta_cohort = 1` (Phase 171 column on `promo_codes`). The code grants Researcher tier for 180 days. Defaults are pre-filled; admin can override.

2. Signup with promo URL (`signup.html?promo=BETA-MMSTUDIO-2026A`): the front-end reads the `promo` URL param, validates the format, shows a banner "Promo code BETA-MMSTUDIO-2026A will be

applied after you create your account." User signs up. After signup, the front-end POSTs to `/api/promo/redeem.php` with the code.

3. Redemption (`/api/promo/redeem.php`): the endpoint applies the tier grant, writes a `promo_redemptions` row, increments `uses_count`, and returns `is_beta_cohort: true` in the response.

4. Routing (`signup.html` and `login.html`): both pages check `data.is_beta_cohort` after the signup or login response. When true, redirect destination is `app-2026.html?draft=1#/mm`. When false, `app.html` (the legacy app). This means a beta user who signs up *or logs in later* lands on MM Studio every time.

The same routing logic lives on the Google auth path (`/api/auth/google.php` returns `is_beta_cohort`, `login.html`'s `handleGoogleCredential` uses it).

URLs for the cohort

- **First-time signup:**
`https://relichecksurvey.com/signup.html?promo=BETA-MMSTUDIO-2026A`
- **Returning login:** `https://relichecksurvey.com/login.html`

Both land at MM Studio for beta-cohort users.



11. Operational conventions and "rules"

These came out of the work and should hold for any future MM Studio change.

Migration files

- One file per phase: `db/schema_phase{N}.sql`.
- Every file starts with `USE dbs15641829;` (mandatory — phpMyAdmin throws #1046 otherwise).
- Use `INFORMATION_SCHEMA` guards to keep migrations idempotent so a re-run is safe.
- End every migration with verification queries (`DESCRIBE`, `COUNT`) so the operator sees what landed.

Endpoint guards

Every MM endpoint must guard against missing companion tables. The pattern: `SHOW TABLES LIKE '...'` + try/catch around every optional read. Missing data should return null or an empty payload, never an HTTP 500 with HTML.

Dropbox sync

The project folder at `/Volumes/Doc Drive/Dropbox/Claude Work/Projects/relicheck/` auto-syncs to the IONOS host every ~15 seconds. **Skip the FileZilla upload step** in build handoffs for this project. Reference the file paths Claude saved if useful, but do not ask Donald to upload.

Build handoff format

Three sections in order: (1) files saved with local paths only, no "do not upload"; (2) SQL schema inline as a copy-paste block in the chat reply; (3) server-side verification steps.

Voice rules

- Avoid personification in writing
- Avoid unintentional anaphora (consecutive sentences starting with the same word)
- Avoid em dashes
- Use on-screen labels verbatim in step-by-step instructions

Diagnostics

- Never ask Donald to paste JavaScript in DevTools Console. Use only visible UI, FileZilla columns, phpMyAdmin SQL, and Network tab reading. Bake diagnostics into the UI itself instead.

Verification

Concrete three-block recipe per change: (a) endpoint reachability check, (b) DB / data check, (c) end-to-end smoke test.



12. Known limits and edge cases

Fake email signups crash on the verify-email send. If a user types `fake@nothing.com` and signs up, the post-signup `relicheck_email_dispatch('customer.welcome.verify_email', ...)` call may throw silently. The try/catch in `signup.php` swallows it but the front-end can show a "Network error" banner if the response shape is unexpected. Real emails work cleanly. This is acceptable for the beta cohort; production may want a friendlier error path.

Question column auto-detection is permissive. The studio looks for a header matching a regex like `(question|q|item)([_\-\s]?id)?` to auto-pick the `question_id` column. Header names like `Q01_text` may be ambiguous. Users can always manually remap.

The Builder uses random discovery sampling. Two runs on the same dataset may surface slightly different theme sets. The studio's guidance to "run it again" if a theme list looks off is honest about this.

Joint Display assumes coded responses exist. If the user opens the Joint Display tab before running the Builder, an empty-state card explains they need to code first. This is from the Phase 76 fix.

Per-question theming requires a question_id column. Without one, the studio treats all open-ended responses as a single bucket. Surveys with multiple open-ended items but no question identifier in the data should add one (Excel: insert a column with values like "Q01" per question's responses) before upload.

The legacy `is_beta_cohort` column on `promo_codes` is required for the beta routing to work. If `schema_phase171.sql` isn't applied, the `try/catch` in `redeem.php` falls back to the older `SELECT`, which means `is_beta_cohort` is always false, which means beta users land on `app.html` not MM Studio.

Existing projects from before Phase 170 were backfilled to `skipped_legacy`. They never see the wizard. New projects always go through it.



13. What is shipped vs. preview

Shipped (closed beta)

- v2 wizard (3 outer steps + sub-steppers)
- All eight analyze tabs (Categories, Dataset, Analysis, Joint Display, Integration, Strength Check, Report) plus scores+comments Score-to-Theme and Alignment
- Quality Brief (Phase 6)
- Builder with two-pass theme discovery (Phase 7-9)
- Variable Builder + IV/DV roles (Phase 10-11)
- Clustering (Phase 12)
- Excel + SPSS exports (Phase 13)
- Analysis suggestions + run (Phase 14)

- Joint Display + Integration paragraphs (Phase 15-16)
- Strength Check (Phase 17)
- Editable Report with version history + rewrite (Phase 18-19)
- DOCX + PDF export (Phase 20)
- Templates (Phase 20 Phase E)
- Walkthrough card on project shell
- Recommended order chip strip on analyze tabs
- Feedback widget (floating button + admin inbox)
- Admin MM Studio activity dashboard
- Admin Beta cohort code provisioning + roster view
- Login / signup routing for beta users to MM Studio
- Beta user guide (HTML + PDF, with placeholder screenshots)

Preview / not yet shipped

- **HRIS integration.** Server scaffolding exists for BambooHR, Workday, Rippling. The tier flag is `false` so users cannot access it. Endpoints return scaffolding data when `HRIS_LIVE_MODE` is undefined. Shipping HRIS requires both a working live mode and an in-app settings panel; neither is done. Decision documented as Track A11.
- **Magic-link beta provisioning.** Alternate to the promo code path; not built. The promo path is what's live.
- **Per-cohort analytics drill-down.** The MM Studio activity admin view shows aggregates and a per-user roll-up but no per-project drill-down. Sufficient for a 10–20 person cohort.
- **Real screenshots in the beta guide.** The HTML and PDF guide reference three named PNGs (`mm-guide-phase1.png`, `mm-guide-phase2.png`, `mm-guide-phase3.png`). They are not in the

project folder. Placeholders show in their place. Real screenshots can be dropped in any time; the guide picks them up automatically.

- **Beta guide content refresh.** The guide still describes the legacy "Phase 1/2/3" structure inside a project. The new "Title → Your Data → Frame Your Study" wizard is not reflected. Content is technically still correct (the post-wizard work has three stages too) but the framing is off.



14. Path forward after closed beta

When the cohort produces enough signal to justify a public launch:

Likely fixes informed by feedback:

- 1 Tighten the Builder's discovery sample for more stable theme sets across runs.
- 2 Reduce the prerequisite knowledge for non-doctoral users (the studio currently assumes IV/DV / Likert / mixed-methods vocabulary).
- 3 Move HRIS to live mode with a real settings panel.
- 4 Refresh the beta guide to match the v2 wizard and embed real screenshots.

Likely additions:

- 1 Per-project drill-down on the admin activity view (see exactly which stage a project is stuck at).
- 2 Manual cohort account provisioning UI (in case promo codes aren't the right path for some institutions).
- 3 Per-template marketplace if templates become a thing users want to share.

Likely retirement:

- 1 The legacy `/api/mm/wizard.php` endpoint (kept for backward compat with old front-end calls). Once no project is on a pre-Phase-170 row, this can go.

Architecture watch:

The `mm_projects.user_id` column is intentionally `user_id` (not `owner_id` like the `surveys` table, which was renamed in Track A138). A future sweep may want to rename for consistency. Test data and projects will need the migration too.



End of MM Studio Bible

Last full rebuild: 2026-05-20, end of closed-beta launch sprint. Project root: `/Volumes/Doc Drive/Dropbox/Claude Work/Projects/relicheck/` Domain: `relichecksurvey.com` Database: `dbs15641829` Maintainer: Donald Easton-Brooks